

Clase 024 — Arquitectura de computadores: CPU, registros y memoria

Parte: **0 — Fundamentos y prerequisites** · Fuente: *Jon Erickson, Hacking: The Art of Exploitation* 🕒
Duración estimada: **110 min** · Nivel: **Fundamentos**

🎯 Objetivo

Entender cómo funciona una CPU a bajo nivel: registros, la pila de llamadas, el ciclo de ejecución y el ensamblador básico. Este conocimiento es el requisito previo indispensable para la explotación binaria, la ingeniería inversa y el análisis de malware que verás en partes avanzadas del programa.

📖 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Describir** los registros clave de x86-64 y su función.
- 2. **Explicar** el ciclo fetch-decode-execute.
- 3. **Trazar** el uso de la pila en una llamada a función.
- 4. **Leer** ensamblador básico y relacionarlo con código C.
- 5. **Conectar** estos conceptos con vulnerabilidades de memoria.

📖 Temas

#	Tema	Por qué importa
1	CPU y ciclo de ejecución	Cómo se ejecutan instrucciones
2	Registros x86-64	RAX, RBX, RSP, RBP, RIP...
3	La pila (stack)	Llamadas, retorno, variables locales
4	Convención de llamada	Cómo se pasan argumentos
5	Ensamblador básico	mov, push, pop, call, ret
6	Endianness	Orden de bytes en memoria
7	Del C al ASM	Compilar y desensamblar
8	Relevancia en exploiting	RIP, overflow, ASLR/DEP

📖 Definiciones y características

- **Registro:** memoria interna y rapidísima de la CPU. Clave: **RIP** apunta a la siguiente instrucción; controlarlo = controlar la ejecución.

- **RSP / RBP**: puntero de pila y de marco. Clave: delimitan el marco de la función actual; centrales en overflows de stack.
- **Pila (stack)**: estructura LIFO que crece hacia direcciones bajas en x86. Clave: guarda dirección de retorno y locales.
- **call / ret**: `call` apila la dirección de retorno y salta; `ret` la desapila a RIP. Clave: sobrescribir esa dirección desvía el flujo.
- **Endianness**: orden de bytes (x86 es little-endian). Clave: al escribir direcciones en un exploit hay que respetarlo.
- **Convención de llamada (System V)**: argumentos en RDI, RSI, RDX, RCX, R8, R9. Clave: saber dónde están los parámetros al leer ASM.

Herramientas y preparación

En Linux/Kali: `gcc`, `objdump`, `gdb` (idealmente con **GEF** o **pwndbg**), `readelf`. Instala GEF:

```
bash -c "$(curl -fsSL https://gef.blah.cat/sh)"
```

Trabaja en tu VM de laboratorio. Un programa C sencillo servirá para compilar, desensamblar y depurar.

Laboratorio guiado

1. **Compilar y desensamblar**. Crea `suma.c` con una función `suma(a,b)` y `main`:

```
bash gcc -O0 -g suma.c -o suma objdump -d -M intel suma | sed -n '/<suma>:/,/ret/p'
```

Identifica `push rbp`, `mov rbp, rsp`, las operaciones y `ret`. 2. **Registros en gdb**:

```
bash gdb ./suma (gdb) break suma (gdb) run (gdb) info registers rdi rsi rsp rbp rip
```

Observa los argumentos en RDI/RSI (System V). 3. **Ver la pila**. En el breakpoint, examina la memoria de la pila:

```
text (gdb) x/8gx $rsp
```

Localiza la dirección de retorno guardada. 4. **Seguir call/ret**. Ejecuta paso a paso (`stepi`) y observa cómo `ret` restaura RIP desde la pila. 5. **Endianness**. Escribe un entero en memoria y examínalo byte a byte para comprobar el orden little-endian. 6. **Conexión con exploiting** (conceptual): razona cómo un buffer local demasiado grande podría sobrescribir la dirección de retorno guardada en la pila, y qué mitigaciones (canario, ASLR, NX/DEP) lo impiden.

 **Nota ética**: la explotación real de vulnerabilidades se practica en partes avanzadas y **solo** en binarios de laboratorio propios o retos autorizados (CTF, VulnHub). Aquí se estudia la teoría.

Ejercicios

1. Enumera 6 registros de x86-64 y describe la función de cada uno.
2. Explica qué guarda la pila cuando se llama a una función y en qué orden.
3. Desensambla una función y anota a qué línea de C corresponde cada instrucción clave.

4. Escribe en little-endian los bytes de la dirección `0x00401136`.
5. Explica cómo un overflow de un buffer en la pila puede alterar RIP.
6. Investiga qué protegen ASLR, NX/DEP y los stack canaries, cada uno contra qué.

Reto verificable

Toma un programa C con una función que reciba argumentos y realice un cálculo. Compíllalo, desensámbalo y, con gdb, produce un "mapa" anotado que muestre: los registros con los argumentos al entrar en la función, la ubicación de la dirección de retorno en la pila, y la instrucción `ret` que la consume. Explica en qué punto un atacante intentaría intervenir.

Criterio de aceptación: el mapa identifica correctamente los registros de argumentos según System V, señala la dirección de retorno en el volcado de la pila (`x/gx $rsp`) y explica coherentemente la relación entre sobrescribir esa dirección y el control de RIP. Reproducible con los mismos comandos de gdb.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El desensamblado no se parece al C	Optimizaciones. Compila con <code>-O0</code> para ver una traducción directa.
gdb no muestra símbolos	Falta información de depuración. Compila con <code>-g</code> .
Direcciones cambian en cada ejecución	ASLR activo. Para estudiar, desactívalo en el lab (<code>setarch -R</code>) con cuidado.
Confundir el orden de bytes	x86 es little-endian; escribe las direcciones al revés byte a byte.
Sintaxis AT&T confusa	Usa <code>-M intel</code> en <code>objdump</code> y <code>set disassembly-flavor intel</code> en gdb.

Preguntas frecuentes

? ¿Necesito ser experto en ensamblador? No para esta clase: basta reconocer patrones (prólogo/epílogo de función, `call/ret`) y saber leer registros. La maestría se construye en las partes de exploiting e ingeniería inversa.

? ¿Por qué x86-64 y no ARM? x86-64 domina el escritorio/servidor y casi toda la literatura de exploiting parte de él. Los conceptos (registros, pila, convención de llamada) se trasladan a ARM con cambios de nombres.

? ¿Qué es RIP y por qué es tan importante? Es el puntero de instrucción: apunta a lo que se ejecutará a continuación. Todo el arte del control-flow hijacking consiste en lograr que RIP apunte donde quiere el atacante.

? ¿Estas mitigaciones (ASLR, DEP) hacen imposible el exploiting? Lo dificultan mucho, no lo impiden: existen técnicas (ROP, infoleaks) para eludirlas. Verlas ahora te prepara para entender esa carrera armamentística.

Referencias

- Jon Erickson, *Hacking: The Art of Exploitation* (No Starch Press).
- Intel 64 and IA-32 Architectures Software Developer's Manual — <https://www.intel.com/sdm>
- System V AMD64 ABI — <https://gitlab.com/x86-psABIs/x86-64-ABI>
- GEF (GDB Enhanced Features) — <https://hugsy.github.io/gef/>

Siguiente clase

Clase 025 - Etica, legalidad, alcance y divulgacion responsable